

dgaiot 本体论白皮书

工业物联网 DLAS 架构 · 本体驱动 AI 编程

dgaiot 项目组*

2026 年 7 月

*<https://gitee.com/dgaiot/dgaiot>

目录

1	核心原则	3
1.1	本体是唯一真相源	3
1.2	生产环境运行确定性代码	3
1.3	修正本体，不修正代码	3
2	DLAS 架构	3
2.1	Data 层	3
2.2	Logic 层	3
2.3	Action 层	4
2.4	Security 层	4
3	本体驱动 AI 编程	4
3.1	闭环流程	4
3.2	本体 → 代码示例	4
4	关系与约束	4
4.1	关系 = 消息路由	4
4.2	约束 = 规则编译	5
5	自调节闭环	5
6	验证结果	5
7	结论	5

1 核心原则

1.1 本体是唯一真相源

本体定义工业系统的完整语义模型。物模型定义”是什么”，拓扑定义”在哪”，通道定义”怎么存”，授权定义”谁能访问”。

1.2 生产环境运行确定性代码

开发时 AI 编程(读本体 → 写代码)。运行时确定性执行(gen_statem·MQTT·TDengine)。没有 AI 推理，没有概率，没有黑盒。

1.3 修正本体，不修正代码

日志说温度 82°C 未告警？检查 gen_statem 是否收到消息（运行时问题）→ 检查 condition 是否正确编译（AI 生成问题）→ 最后才怀疑本体（极少）。

2 DLAS 架构

DLAS 是 Data·Logic·Action·Security 四层架构。

2.1 Data 层

- Parse/PG: 23 类 JSONB 存储 (Site, Gateway, Device, Point, Product, ...)
- TDengine: __{ChannelId}.__{ProductId}, devaddr NCHAR(50) 强制存在
- ETS: 3 表 (model, instance, rules), O(1) 查找, <1μs
- 15 张 __Join 关系表 (rules:__Role 85,497 行, menus:__Role 8,492 行...)

2.2 Logic 层

- dgiot_ontology: load_model + spawn_instance + ETS + push_point
- dgiot_thing: decoder (二进制 → 物模型) + check_value (阈值校验)
- dgiot_product: parse_frame + get_productSecret + lookup_prod
- dgiot_role: childrole 递归 + ACL/CLP
- dgiot_parse_id: 30+ MD5 确定性编码函数

2.3 Action 层

每个物理设备对应一个 Erlang gen_statem 进程:

```
init -> authenticate -> online -> {normal, alarm, offline}
online + {data, T=82.3} -> R_HIGH_TEMP -> alarm
alarm + heartbeat -> online (recovery)
online + heartbeat_missed -> offline
```

211 台设备 = 211 个 gen_statem 进程, 总内存 <1MB。

2.4 Security 层

三层 MQTT 授权: 设备 (ProductSecret)、用户 (Token+Role+ACL)、超级用户 (本地免检)。

3 本体驱动 AI 编程

3.1 闭环流程

企业需求 → 本体建模 → AI 生成代码 → 测试验证 → 确定性生产 → 日志反馈 → 修正本体。

3.2 本体 → 代码示例

thing_model.json 的一个 property:

```
{"name": "油压", "dataForm": {"address": "40300", "protocol": "modbus", "originaltype": "float32_AB"}, "alarm": {"high": 3.0}}
```

AI 自动生成:

- Modbus: readHoldingRegisters(40300, 2, float32_AB)
- MQTT topic: dgaiot/oil_field_01/gw_131/rtu_001/oil_pressure/data
- TDengine: INSERT INTO _85ef6b7459._2de1b3e1b8 VALUES (NOW, v, 192)
- gen_statem guard: oil_pressure > 3.0 → alarm L2

4 关系与约束

4.1 关系 = 消息路由

Erlang 中所有关系都通过消息传递实现。TCP Worker → Decoder → Bridge → MQTT → Shadow → Parser。整条链都是异步 cast 消息。

4.2 约束 = 规则编译

`thing_model.rules[]` 在 `load_model` 时编译为 `gen_statem guard` 子句。流式规则编译为 `EdgeStreamEngine` 滑动窗口。PHM 规则编译为 `TDengine SQL` 窗口查询。

5 自调节闭环

1. `alter_table` 自同步：物模型加了新属性 → 自动检测 → `ADD COLUMN`
2. ETS 缓存自预热：`register(point)` 时异步计算 `path` 并缓存
3. 影子自恢复：`heartbeat miss 30s`→`offline`，`heartbeat` 恢复 →`online`
4. 物模型自修正：日志累积误报 → AI 建议修改阈值 → 人工确认 → 重新生成
5. 通道自切换：`DTU GPRS` 丢包 >30% → 自动切 4G 备用通道

6 验证结果

7 结论

dgaIoT 提出了一种本体驱动 AI 编程的工业物联网架构。本体是唯一真相源，AI 在开发阶段将本体编译为确定性代码，生产环境运行的是 `gen_statem` + `MQTT` + `TDengine` 的确定性管道。DLAS 四层架构、15 张关系表、30+ 确定性编码函数、5 种自调节机制已通过 `Kylin-DMZ` 实测验证。

表 1: 全链路验证矩阵

测试项	方法	结果
MQTT pub/sub	Kylin-DMZ 实机	3 发 3 收, DLAS topic
gen_statem Shadow	Erlang 编译运行	init→online→alarm→online
TDengine 存储	源码对齐建表	INSERT 15 点, SELECT 15 行
dlink 设备授权	ClientID={PID}_DevId	{DevId}全部 OK